

Scripts Execution & Logic Explanation

Explanation of the solution to the streaming layer problem

Code Implementation:

- Import the necessary libraries related to PySpark, Spark Streaming, classes and functions definitions provided as template code (dao.py, geo_map.py and rules.py).

```

import datetime
from datetime import datetime
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, from_json, to_timestamp, udf
from pyspark.sql.types import StructType, StructField, StringType, IntegerType, LongType, TimestampType
from db.dao import HBaseDao
from db.geo_map import GEO_Map
from rules.rules import is_score_rejected, is_ucl_exceeded, is_zipcode_invalid

```

- Define the required utility functions and UDF on top of it.
 - get_distance():** This function is to calculate the distance between current and previous locations where transactions happened based on its latitude and longitude. It instantiates and calls **GEO_Map** class (**geo_map.py**) methods to get the latitude and longitude attributes value and calculate the distance in km.

db/geo_map.py

```

import math
import pandas as pd

class GEO_Map():
    """
    It hold the map for zip code and its latitude and longitude
    """
    __instance = None

    @staticmethod
    def get_instance():
        """ Static access method. """
        if GEO_Map.__instance == None:
            GEO_Map()
            return GEO_Map.__instance
        return GEO_Map.__instance

    def __init__(self):
        """ Virtually private constructor. """
        if GEO_Map.__instance != None:
            raise Exception("This class is a singleton!")
        else:
            GEO_Map.__instance = self
            self.map = pd.read_csv("uszip.csv", header=None, names=['A', 'B', 'C', 'D', 'E'])
            self.map['A'] = self.map['A'].astype(str)

    def get_lat(self, pos_id):
        return self.map[self.map.A == pos_id].B

    def get_long(self, pos_id):
        return self.map[self.map.A == pos_id].C

    def distance(self, lat1, long1, lat2, long2):
        theta = long1 - long2
        dist = math.sin(self.deg2rad(lat1)) * math.sin(self.deg2rad(lat2)) + math.cos(self.deg2rad(lat1)) * math.cos(self.deg2rad(lat2)) * math.cos(self.deg2rad(theta))
        dist = math.acos(dist)
        dist = self.rad2deg(dist)
        dist = dist * 60 * 1.1515 * 1.609344
        return dist

    def rad2deg(self, rad):
        return rad * 180.0 / math.pi

    def deg2rad(self, deg):
        return deg * math.pi / 180.0

```

driver.py

```
from rules.rules import is_score_rejected, is_ucl_exceeded, is_zipcode_invalid

# Utility function to calculate distance between last two locations where transactions happened
def get_distance(trans_postcode, lkp_postcode):
    """
    Calculate the distance between current and previous locations where transactions happened
    Input Arguments: trans_postcode (postcode from card_transactions real time data - Kafka topic in JSON format)
                    lkp_postcode (postcode from card_lookup table (Hbase) of the same card_id from card_transactions real time
    data).
    """
    # Instantiate GEO_Map class object
    geo_obj = GEO_Map.get_instance()
    lat1 = float(geo_obj.get_lat(trans_postcode).values[0])
    long1 = float(geo_obj.get_long(trans_postcode).values[0])
    lat2 = float(geo_obj.get_lat(lkp_postcode).values[0])
    long2 = float(geo_obj.get_long(lkp_postcode).values[0])
    return geo_obj.distance(lat1, long1, lat2, long2)
```

- b) **get time diff secs()**: This function calculates the time difference between current and previous transactions timestamp.

driver.py

```
# Utility function to calculate time difference between last two transactions happened
def get_time_diff_secs(trans_transactiondt, lkp_transactiondt):
    """
    Calculate the time difference between current and previous transactions timestamp.
    Input Arguments: trans_transactiondt (transaction_dt from card_transactions real time data - Kafka topic in JSON format)
                    lkp_transactiondt (transaction_dt from card_lookup table (Hbase) of the same card_id from card_transactions
    real time data).
    """
    return (trans_transactiondt - lkp_transactiondt).total_seconds()
```

- c) **insert card transactions hbase()**: This function inserts card transactions real time data (Kafka topic in JSON format) into card_transactions HBase table. It first instantiates HbaseDao class object to connect to HBase database and then call required methods to insert the data in card transactions table.

driver.py

```
# Write card transactions final data to HBase table
def insert_card_transactions_hbase(transaction_data):
    """
    Insert card transactions real time data (Kafka topic in JSON format) into card_transactions hbase table.
    Input Arguments: transaction_data (list object of card transactions real time data - Kafka topic in JSON format).
    """
    # Instantiate HBaseDao class object to connect to HBase
    hbase_obj = HBaseDao.get_instance()
    trans_key = '|'.join([str(v) for v in transaction_data])
    data = {'transaction_data:card_id':bytes(str(transaction_data[0]), encoding='utf8'),
            b'transaction_data:member_id':bytes(str(transaction_data[1]), encoding='utf8'),
            b'transaction_data:amount':bytes(str(transaction_data[2]), encoding='utf8'),
            b'transaction_data:pos_id':bytes(str(transaction_data[3]), encoding='utf8'),
            b'transaction_data:postcode':bytes(str(transaction_data[4]), encoding='utf8'),
            b'transaction_data:transaction_dt':bytes(str(transaction_data[5]), encoding='utf8'),
            b'transaction_data:status':bytes(str(transaction_data[6]), encoding='utf8')}
    hbase_obj.write_data(bytes(trans_key, encoding='utf8'), data, 'card_transactions')
```

db/dao.py

```
import happybase

class HBaseDao:
    """
    Dao class for operation on HBase
    """
    __instance = None

    @staticmethod
    def get_instance():
        """ Static access method. """
        if HBaseDao.__instance == None:
            HBaseDao()
            return HBaseDao.__instance

    def __init__(self):
        if HBaseDao.__instance != None:
            raise Exception("This class is a singleton!")
        else:
            HBaseDao.__instance = self
            self.host = '44.214.91.136'
            self.connect()

    def connect(self):
        for i in range(3):
            try:
                self.pool = happybase.ConnectionPool(size=3, host=self.host, port=8090)
                break
            except:
                print("Exception in connecting HBase")

    def get_data(self, key, table):
        for i in range(2):
            try:
                with self.pool.connection() as connection:
                    t = connection.table(table)
                    row = t.row(key)
                    return row
            except:
                self.reconnect()

    def write_data(self, key, row, table):
        error = None
        for i in range(2):
            try:
                with self.pool.connection() as connection:
                    t = connection.table(table)
                    return t.put(key, row)
            except Exception as e:
                error = e
                self.reconnect()
        raise Exception(key + str(e))

    def reconnect(self):
        self.connect()
```

- d) **update_card_lookup_hbase()**: This function updates postcode and transaction_dt attributes in card lookup table HBase for which transactions are treated as GENUINE. It first instantiates HbaseDao class object to connect to HBase database and then call required methods to update the card lookup data.

driver.py

```
# Update latest postcode and transaction_date in card lookup HBase table
def update_card_lookup_hbase(trans_card_id, status_flag, lkp_data):
    """
    Update postcode and transaction_dt attributes in card lookup table HBase for which transactions are treated as GENUINE.
    Input Arguments: trans_card_id
                    status_flag
                    lkp_data (List object of postcode and transaction_dt card transactions real time data - Kafka topic in JSON
    Format).
    """
    # Instantiate HBaseDao class object to connect to HBase
    hbase_obj = HBaseDao.get_instance()
    if status_flag == 'GENUINE':
        key = bytes(str(trans_card_id), encoding='utf8')
        data = {'lkp_info:postcode': bytes(str(lkp_data[0]), encoding='utf8'),
                b'lkp_info:transaction_dt': bytes(str(lkp_data[1]), encoding='utf8')}
        hbase_obj.write_data(key, data, 'card_lookup')
```

- e) **check status()**: This function is to detect the fraudulent of the live transactions based on the rules defined (mentioned in the problem approach) and lookup data prepared as part of batch processing.
- rules/rules.py**

```

1. is_score_rejected() is to check the credit score and see whether it is less than threshold value for rejection.
def is_score_rejected(score):
    """
    This function is to get the credit score of the particular card which transaction happened and see whether it has greater than or equal to 200. If not then it is labelled as Fraud.
    Input Arguments: score (credit_score from card_transactions real time data - Kafka topic in JSON format)
    """
    if score < 200:
        return True
    return False

# 2. is_ucl_exceeded() is to check whether the amount exceeds the upper control limit of the transaction to be done.
def is_ucl_exceeded(amount, ucl):
    """
    This function is to check whether the amount exceeds the upper control limit of the transaction to be done. If it is exceeded then it is labelled as Fraud.
    Input Arguments: amount (card_transactions real time data - Kafka topic in JSON format)
                    ucl (UCL attribute value from card_lookup table - HBase)
    """
    if amount > ucl:
        return True
    return False

# 3. is_zipcode_invalid() is to check whether the travelling speed between current and last transaction happened is valid or not.
def is_zipcode_invalid(distance, time_diff_secs):
    """
    This function is to measure the speed in Kilometers/second travelled by the customer between two locations where customer made the recent two transactions (current and previous).
    Usually commercial airplanes travel with an average speed of around 800-950 Kilometers/hour. We considered the speed threshold value as 900 Kilometers/hour and converted it to seconds which is (900/3600) = 0.25 Kilometers/second. If the measure speed between two locations is greater than 0.25 Kilometers/second then it is labelled as Fraud.
    Input Arguments: distance (distance covered between two locations of recent two transactions)
                    time_diff_secs (Time difference between last two transactions happened)
    """
    if time_diff_secs == 0:
        if distance != 0:
            return True
    else:
        speed_kmps = distance / time_diff_secs
        if speed_kmps > 0.25:
            return True
    return False

```

driver.py

```
# Utility function to detect fraudulent
def check_status(trans_card_id, trans_member_id, trans_amount, trans_pos_id, trans_postcode, trans_transactiondt):
    """
    This function is to detect the fraudulent of the live transactions based on the rules defined and lookup data prepared as part of batch processing.
    The following three rules to be validated:
    1. Check credit score
    2. Check UCL threshold with amount
    3. Measure the speed travelled in Kilometers/second between the locations of last two transactions.
    If any of the above rule fails then the transaction is labelled as FRAUD. Otherwise it is labelled as GENUINE.
    Input Arguments: trans_card_id (card id attribute from card_transactions real time data - Kafka topic in JSON format)
                    trans_member_id (member id attribute from card_transactions real time data - Kafka topic in JSON format)
                    trans_amount (transaction amount from card_transactions real time data - Kafka topic in JSON format)
                    trans_pos_id (pos_id from card_transactions real time data - Kafka topic in JSON format)
                    trans_postcode (postcode from card_transactions real time data - Kafka topic in JSON format)
                    trans_transactiondt (transaction_dt from card_transactions real time data - Kafka topic in JSON format)
    """
    status_flag = 'GENUINE'
    # Instantiate HBaseDao class object to connect to HBase
    hbase_obj = HBaseDao.get_instance()
    lkp_row = hbase_obj.get_data(str(trans_card_id), 'card_lookup')
    if lkp_row == {}:
        pass
    else:
        lkp_score, lkp_postcode, lkp_transactiondt, lkp_ucl = [val.decode('utf8') for val in list(lkp_row.values())]
        lkp_score = int(lkp_score)
        lkp_postcode = int(lkp_postcode)
        lkp_transactiondt = datetime.strptime(lkp_transactiondt, '%Y-%m-%d %H:%M:%S')
        lkp_ucl = float(lkp_ucl)

        # Check transactions rules to determine credit score, UCL and zipcode distance.
        if is_score_rejected(lkp_score):
            status_flag = 'FRAUD'
        elif is_ucl_exceeded(trans_amount, lkp_ucl):
            status_flag = 'FRAUD'
        elif is_zipcode_invalid(get_distance(str(trans_postcode), str(lkp_postcode)), get_time_diff_secs(trans_transactiondt, lkp_transactiondt)):
            status_flag = 'FRAUD'
        else:
            pass

    # Call insert_card_transactions_hbase() function to insert the card transaction data with status derived
    insert_card_transactions_hbase([trans_card_id, trans_member_id, trans_amount, trans_pos_id, trans_postcode, trans_transactiondt, status_flag])
    # Call update_card_lookup_hbase() to insert/update postcode and transaction_dt attributes in card_lookup HBase table
    update_card_lookup_hbase(trans_card_id, status_flag, [trans_postcode, trans_transactiondt])

    return status_flag

# Create fraud detection UDF on top of status utility function
fraud_detection = udf(check_status, StringType())
```

- f) Define UDF **fraud_detection** on top of **check_status()** utility function that to be used in spark streaming dataframe for deriving “status” attribute during real time. As part of this UDF, it insert the records into card transactions HBase table and update the postcode and transaction date in card lookup HBase table (as shown in **check_status()** driver.py screenshot above).

driver.py

```
# Create fraud detection UDF on top of status utility function
fraud_detection = udf(check_status, StringType())
```

- Create a Spark Session which is an entry point to spark application and set the log level to 'ERROR'.

driver.py

```
# Create a spark session
spark = SparkSession \
    .builder \
    .appName("Credit Card Fraud Detection App") \
    .getOrCreate()

# Set log level to ERROR
spark.sparkContext.setLogLevel('ERROR')
```

- Read the card transactions real time data from Kafka source topic (transactions-topic verified).

driver.py

```
# Read card transactions from Kafka topic : transactions-topic-verified
card_trans_source = spark \
    .readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "18.211.252.152:9092") \
    .option("subscribe", "transactions-topic-verified") \
    .option("startingOffsets", "earliest") \
    .load()
```

- Define the proper data structure or schema and assign it to the source data read from Kafka topic.

driver.py

```
# Defining the Data Structure for source card transactions (json format).
data_structure = StructType([
    StructField("card_id", LongType()),
    StructField("member_id", LongType()),
    StructField("amount", IntegerType()),
    StructField("pos_id", LongType()),
    StructField("postcode", IntegerType()),
    StructField("transaction_dt", StringType())
])

# Converting source data from json format to dataframe (table).
card_trans_formatted = card_trans_source \
    .select(from_json(col("value").cast("string"), data_structure).alias("card_transactions")) \
    .select("card_transactions.*") \
    .withColumn("transaction_dt", to_timestamp(col('transaction_dt'), 'dd-MM-yyyy HH:mm:ss')) \
    .select("card_id",
           "member_id",
           "amount",
           "pos_id",
           "postcode",
           "transaction_dt"
    )
```

- Add a new column “status” by calling the fraud_detection UDF defined above. This UDF will also take of inserting and updating the records in card transactions and card lookup HBase tables.

driver.py

```
# Derive status attribute: GENUINE or FRAUD based on rules defined
card_trans_transformed = card_trans_formatted \
    .withColumn("status", fraud_detection(card_trans_formatted.card_id, card_trans_formatted.member_id, card_tr
ans_formatted.amount, card_trans_formatted.pos_id, card_trans_formatted.postcode, card_trans_formatted.transaction_dt)) \
    .select("card_id",
            "member_id",
            "amount",
            "pos_id",
            "postcode",
            "transaction_dt",
            "status"
            )
```

- Write the final transformed card transactions data with its status in the console for live analysis. It runs as a micro-batch with processing time of every “1 Minute” until it is interrupted/terminated.

driver.py

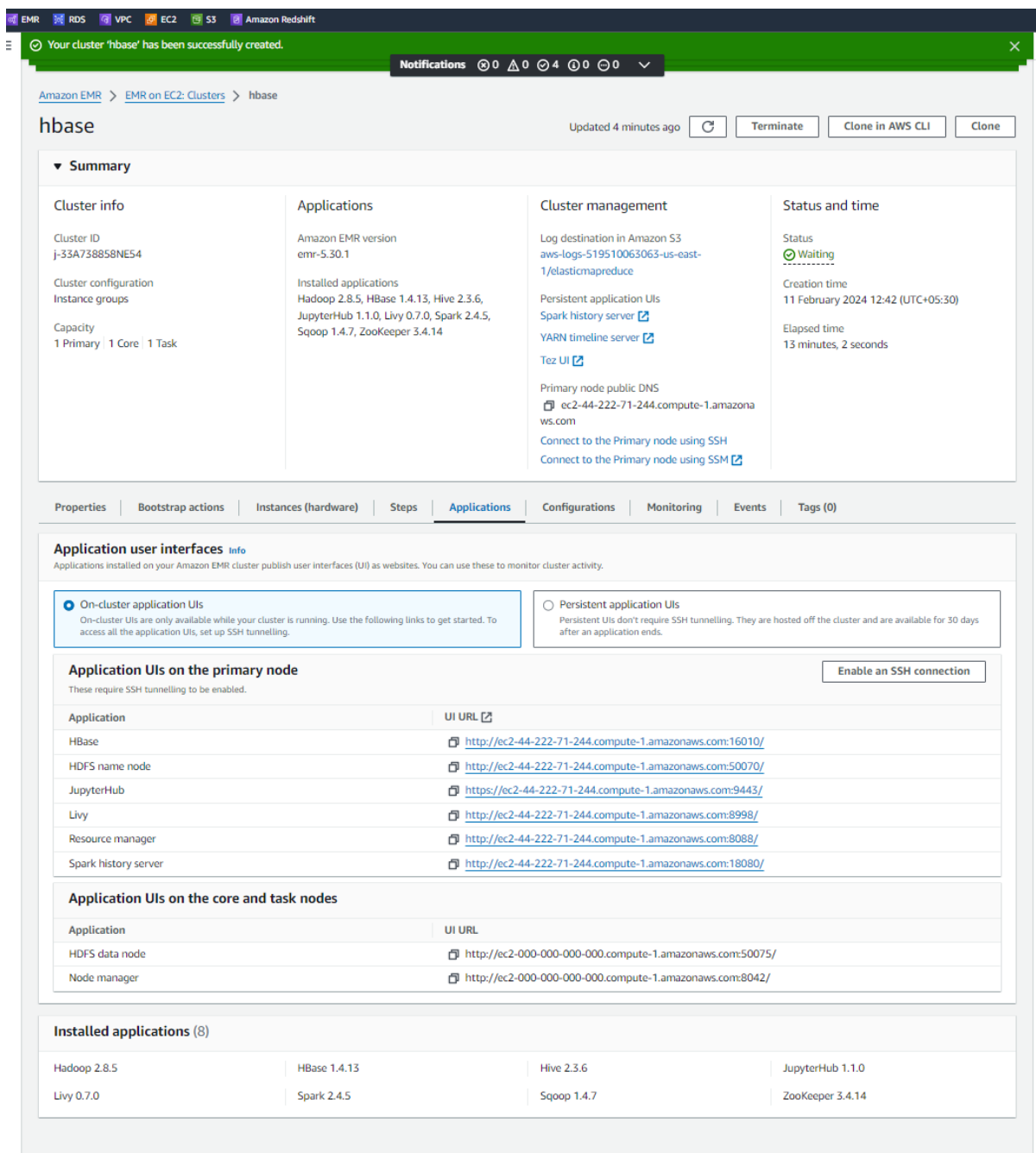
```
# Write the final transformed data to sink
card_trans_sink = card_trans_transformed \
    .writeStream \
    .outputMode("append") \
    .format("console") \
    .option("truncate", "false") \
    .trigger(processingTime="1 minute") \
    .start()

# Await termination
card_trans_sink.awaitTermination()
```

DEPLOYMENT STEPS:

PRE-REQUISITES:

- Create an EMR cluster with Hive, HBase, Sqoop, Spark, Zookeeper applications installed in it as shown below:



The screenshot displays the AWS Management Console interface for an Amazon EMR cluster named 'hbase'. The cluster is in a 'Waiting' state. The console shows various tabs for cluster management, including Summary, Properties, Bootstrap actions, Instances (hardware), Steps, Applications, Configurations, Monitoring, Events, and Tags (0). The 'Applications' tab is currently selected, showing a list of installed applications and their UI URLs.

Cluster Summary:

- Cluster ID:** j-33A738858NE54
- Cluster configuration:** Instance groups
- Capacity:** 1 Primary | 1 Core | 1 Task
- Applications:** Amazon EMR version emr-5.30.1. Installed applications: Hadoop 2.8.5, HBase 1.4.13, Hive 2.3.6, JupyterHub 1.1.0, Livy 0.7.0, Spark 2.4.5, Sqoop 1.4.7, ZooKeeper 3.4.14.
- Cluster management:** Log destination in Amazon S3: aws-logs-519510063063-us-east-1/elasticmapreduce. Persistent application UIs: Spark history server, YARN timeline server, Tez UI.
- Status and time:** Status: Waiting. Creation time: 11 February 2024 12:42 (UTC+05:30). Elapsed time: 13 minutes, 2 seconds.

Application user interfaces:

Applications installed on your Amazon EMR cluster publish user interfaces (UI) as websites. You can use these to monitor cluster activity.

On-cluster application UIs: On-cluster UIs are only available while your cluster is running. Use the following links to get started. To access all the application UIs, set up SSH tunnelling.

Persistent application UIs: Persistent UIs don't require SSH tunnelling. They are hosted off the cluster and are available for 30 days after an application ends.

Application UIs on the primary node: These require SSH tunnelling to be enabled.

Application	UI URL
HBase	http://ec2-44-222-71-244.compute-1.amazonaws.com:16010/
HDFS name node	http://ec2-44-222-71-244.compute-1.amazonaws.com:50070/
JupyterHub	https://ec2-44-222-71-244.compute-1.amazonaws.com:9443/
Livy	http://ec2-44-222-71-244.compute-1.amazonaws.com:8998/
Resource manager	http://ec2-44-222-71-244.compute-1.amazonaws.com:8088/
Spark history server	http://ec2-44-222-71-244.compute-1.amazonaws.com:18080/

Application UIs on the core and task nodes:

Application	UI URL
HDFS data node	http://ec2-000-000-000-000.compute-1.amazonaws.com:50075/
Node manager	http://ec2-000-000-000-000.compute-1.amazonaws.com:8042/

Installed applications (8):

Hadoop 2.8.5	HBase 1.4.13	Hive 2.3.6	JupyterHub 1.1.0
Livy 0.7.0	Spark 2.4.5	Sqoop 1.4.7	ZooKeeper 3.4.14

- Download and install MySQL connector in AWS EMR for performing Sqoop data ingestion from AWS RDS to HDFS.

```
cd /home/hadoop
wget https://de-mysql-connector.s3.amazonaws.com/mysql-connector-
java-8.0.25.tar.gz
tar -xvf mysql-connector-java-8.0.25.tar.
cd mysql-connector-java-8.0.25/
sudo cp mysql-connector-java-8.0.25.jar /usr/lib/sqoop/lib/
```

- Create all the required tables in Hive and HBase such as card_member, member_score, card_transactions (historical data) and card_lookup tables and load the pre-processed data in it as part of batch processing (mid submission).

```
total 8780
drwxr-xr-x 3 hadoop hadoop      149 Apr 27  2021 mysql-connector-java-8.0.25
-rw-rw-r-- 1 hadoop hadoop 4079310 Aug  7  2021 mysql-connector-java-8.0.25.tar.gz
-rwxrwxrwx 1 hadoop hadoop 4829520 Jan 17 07:48 card_transactions.csv
-rwxrwxrwx 1 hadoop hadoop   665 Jan 18 09:39 ingest.sh
-rwxrwxrwx 1 hadoop hadoop  1195 Jan 18 10:29 pre.hql.bak
-rwxrwxrwx 1 hadoop hadoop   314 Jan 18 10:30 setup.txt
-rwxrwxrwx 1 hadoop hadoop  1644 Feb 10 11:53 createHistorical.hql.bak
-rwxrwxrwx 1 hadoop hadoop   407 Feb 10 12:06 loadNosql.sh.bak
-rwxrwxrwx 1 hadoop hadoop  1651 Feb 11 05:50 createHistorical.hql
-rwxrwxrwx 1 hadoop hadoop   640 Feb 11 05:50 createLookupTable.hql.bak
-rwxrwxrwx 1 hadoop hadoop   636 Feb 11 05:50 createLookupTable.hql
-rwxrwxrwx 1 hadoop hadoop   397 Feb 11 05:52 loadNosql.sh
-rwxrwxrwx 1 hadoop hadoop  1193 Feb 11 05:52 pre.hql
drwxrwxrwx 3 hadoop hadoop     17 Feb 11 08:03 python
-rw-rw-r-- 1 hadoop hadoop  21228 Feb 11 08:06 card_member.java
-rw-rw-r-- 1 hadoop hadoop  11329 Feb 11 08:07 member_score.java
[hadoop@ip-172-31-11-213 ~]$
```

sh loadNosql.sh

```
[hadoop@ip-172-31-11-213 ~]$ sh loadNosql.sh
rm: `hdfs:/user/hadoop/hive': No such file or directory
0 row(s) in 2.6030 seconds

Hbase::Table - card_transactions
0 row(s) in 2.5980 seconds

Hbase::Table - card_lookup

Logging initialized using configuration in file:/etc/hive/conf.dist/hive-log4j2.properties Async: true
OK
Time taken: 1.583 seconds
Loading data to table default.card_transactions_stg
OK
Time taken: 2.17 seconds
OK
Time taken: 1.211 seconds
Query ID = hadoop_20240211080541_a4e45c35-dbc1-44b2-a38f-abf210e26cfa
Total jobs = 1
Launching Job 1 out of 1
Status: Running (Executing on YARN cluster with App id application_1707638368249_0002)

-----
VERTICES      MODE      STATUS  TOTAL  COMPLETED  RUNNING  PENDING  FAILED  KILLED
-----
Map 1 ..... container      SUCCEEDED      1          1          0          0          0          0
-----
VERTICES: 01/01 [=====>>>] 100%  ELAPSED TIME: 9.21 s
-----

OK
Time taken: 36.545 seconds
OK
Time taken: 0.314 seconds

Logging initialized using configuration in file:/etc/hive/conf.dist/hive-log4j2.properties Async: true
OK
Time taken: 2.181 seconds
[hadoop@ip-172-31-11-213 ~]$ sh ingest.sh
```

sh ingest.sh

[illegible]

hive -f pre.hql

HBase Shell

```
[root@ip-172-31-14-192 ~]# hbase shell
HBase Shell
Use "help" to get list of supported commands.
Use "exit" to quit this interactive shell.
Version 1.4.13, rUnknown, Fri Apr 17 15:18:24 UTC 2020

hbase(main):001:0> list
TABLE
card_lookup
card_transactions
2 row(s) in 0.1720 seconds

=> ["card_lookup", "card_transactions"]
hbase(main):002:0> █
```

Hive:

show tables;

```
[root@ip-172-31-14-192 ~]# hive
Logging initialized using configuration in file:/etc/hive/conf.dist/hive-log4j2.properties Async: true
hive> show tables;
OK
card_lookup
card_member
card_transactions
card_transactions_stg
last_ten_transactions
member_score
Time taken: 0.551 seconds, Fetched: 6 row(s)
```

- Switch to root user in EMR cluster and install pandas and happybase python packages.

```
sudo su root
yum install gcc
yum install python3-devel
pip install cython
pip install numpy
pip install pandas
pip install happybase
```

```
[root@ip-172-31-11-213 ~]# pip install pandas --use-feature=2020-resolver
WARNING: Running pip install with root privileges is generally not a good idea. Try 'pip3 install --user' instead.
Collecting pandas
  Downloading pandas-1.3.5-cp37-cp37m-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (11.3 MB)
    | 11.3 MB 40.9 MB/s
Requirement already satisfied: pytz>=2017.3 in /usr/local/lib/python3.7/site-packages (from pandas) (2019.3)
Collecting python-dateutil>=2.7.3
  Downloading python-dateutil-2.8.2-py2.py3-none-any.whl (247 kB)
    | 247 kB 52.2 MB/s
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.7/site-packages (from python-dateutil>=2.7.3->pandas) (1.16.0)
Collecting numpy>=1.17.3
  Downloading numpy-1.21.6-cp37-cp37m-manylinux_2_12_x86_64.manylinux2010_x86_64.whl (15.7 MB)
    | 15.7 MB 55.6 MB/s
Installing collected packages: python-dateutil, numpy, pandas
  Attempting uninstall: numpy
    Found existing installation: numpy 1.16.5
    Uninstalling numpy-1.16.5:
      Successfully uninstalled numpy-1.16.5
  WARNING: The scripts f2py, f2py3 and f2py3.7 are installed in '/usr/local/bin' which is not on PATH.
  Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
ERROR: python37-sagemaker-pyspark 1.3.0 requires pyspark==2.3.4, which is not installed.
Successfully installed numpy-1.21.6 pandas-1.3.5 python-dateutil-2.8.2
[root@ip-172-31-11-213 ~]# pip install kafka-python --use-feature=2020-resolver
WARNING: Running pip install with root privileges is generally not a good idea. Try 'pip3 install --user' instead.
Requirement already satisfied: kafka-python in /usr/local/lib/python3.7/site-packages (2.0.2)
[root@ip-172-31-11-213 ~]# pip install pandas --use-feature=2020-resolver
WARNING: Running pip install with root privileges is generally not a good idea. Try 'pip3 install --user' instead.
Requirement already satisfied: pandas in /usr/local/lib64/python3.7/site-packages (1.3.5)
Requirement already satisfied: python-dateutil>=2.7.3 in /usr/local/lib/python3.7/site-packages (from pandas) (2.8.2)
Requirement already satisfied: numpy>=1.17.3 in /usr/local/lib64/python3.7/site-packages (from pandas) (1.21.6)
Requirement already satisfied: pytz>=2017.3 in /usr/local/lib/python3.7/site-packages (from pandas) (2019.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.7/site-packages (from python-dateutil>=2.7.3->pandas) (1.16.0)
[root@ip-172-31-11-213 ~]# pip list
Package Version
-----
aws-cfn-bootstrap 2.0
beautifulsoup4 4.9.0
boto 2.49.0
docutils 0.14
happybase 1.2.0
jmespath 0.9.5
kafka-python 2.0.2
lockfile 0.11.0
lxml 4.5.0
mysqlclient 1.4.2
nltk 3.4.5
nose 1.3.4
numpy 1.21.6
pandas 1.3.5
pip 20.2.2
ply 3.11
py-dateutil 2.2
pystache 0.5.4
python-daemon 2.2.3
python-dateutil 2.8.2
python37-sagemaker-pyspark 1.3.0
pytz 2019.3
PyYAML 5.3.1
setuptools 49.1.3
simplejson 3.2.0
six 1.16.0
```

Script Execution:

- Login to AWS EMR and create the python directory under home directory and place all the executable python files inside it (either using scp command or WinSCP tool) and change its permissions.

```
cd /home/hadoop
```

```
mkdir -p /home/hadoop/python/src
```

```
[hadoop@ip-172-31-14-192 ~]$ ls -lrt
total 4760
-rw-rw-r-- 1 hadoop hadoop 4829520 Jan 17 07:48 card_transactions.csv
-rw-rw-r-- 1 hadoop hadoop 665 Jan 18 09:39 ingest.sh
-rw-rw-r-- 1 hadoop hadoop 1195 Jan 18 10:29 pre.hql.bak
-rw-rw-r-- 1 hadoop hadoop 314 Jan 18 10:30 setup.txt
-rw-rw-r-- 1 hadoop hadoop 1644 Feb 10 11:53 createHistorical.hql.bak
-rw-rw-r-- 1 hadoop hadoop 407 Feb 10 12:06 loadNosql.sh.bak
-rw-rw-r-- 1 hadoop hadoop 1651 Feb 11 05:50 createHistorical.hql
-rw-rw-r-- 1 hadoop hadoop 640 Feb 11 05:50 createLookupTable.hql.bak
-rw-rw-r-- 1 hadoop hadoop 636 Feb 11 05:50 createLookupTable.hql
-rw-rw-r-- 1 hadoop hadoop 397 Feb 11 05:52 loadNosql.sh
-rw-rw-r-- 1 hadoop hadoop 1193 Feb 11 05:52 pre.hql
drwxrwxr-x 3 hadoop hadoop 17 Feb 11 07:27 python
[hadoop@ip-172-31-14-192 ~]$ chmod 777 *
[hadoop@ip-172-31-14-192 ~]$ ls -lrt
total 4760
-rwxrwxrwx 1 hadoop hadoop 4829520 Jan 17 07:48 card_transactions.csv
-rwxrwxrwx 1 hadoop hadoop 665 Jan 18 09:39 ingest.sh
-rwxrwxrwx 1 hadoop hadoop 1195 Jan 18 10:29 pre.hql.bak
-rwxrwxrwx 1 hadoop hadoop 314 Jan 18 10:30 setup.txt
-rwxrwxrwx 1 hadoop hadoop 1644 Feb 10 11:53 createHistorical.hql.bak
-rwxrwxrwx 1 hadoop hadoop 407 Feb 10 12:06 loadNosql.sh.bak
-rwxrwxrwx 1 hadoop hadoop 1651 Feb 11 05:50 createHistorical.hql
-rwxrwxrwx 1 hadoop hadoop 640 Feb 11 05:50 createLookupTable.hql.bak
-rwxrwxrwx 1 hadoop hadoop 636 Feb 11 05:50 createLookupTable.hql
-rwxrwxrwx 1 hadoop hadoop 397 Feb 11 05:52 loadNosql.sh
-rwxrwxrwx 1 hadoop hadoop 1193 Feb 11 05:52 pre.hql
drwxrwxrwx 3 hadoop hadoop 17 Feb 11 07:27 python
[hadoop@ip-172-31-14-192 ~]$
```

- Change the executable files permissions:

```
[hadoop@ip-172-31-14-192 ~]$ cd /home/hadoop/python/src
[hadoop@ip-172-31-14-192 ~]$ chmod 777 python/*
[hadoop@ip-172-31-14-192 ~]$ chmod 777 python/src/*
[hadoop@ip-172-31-14-192 ~]$ chmod 777 python/src/db/*
[hadoop@ip-172-31-14-192 ~]$ chmod 777 python/src/rules/*
driver.py - /home/hadoop/src
```

```
[hadoop@ip-172-31-14-192 ~]$ chmod 777 python/*
[hadoop@ip-172-31-14-192 ~]$ chmod 777 python/src/*
[hadoop@ip-172-31-14-192 ~]$ chmod 777 python/src/db/*
[hadoop@ip-172-31-14-192 ~]$ chmod 777 python/src/rules/*
[hadoop@ip-172-31-14-192 ~]$ cd python/src/
[hadoop@ip-172-31-14-192 src]$ ls -lrt
total 736
-rwxrwxrwx 1 hadoop hadoop 735312 Feb 11 05:27 uszipsv.csv
-rwxrwxrwx 1 hadoop hadoop 2968 Feb 11 05:27 src.zip
-rwxrwxrwx 1 hadoop hadoop 0 Feb 11 05:27 __init__.py
-rwxrwxrwx 1 hadoop hadoop 10131 Feb 11 05:51 driver.py
drwxrwxrwx 3 hadoop hadoop 76 Feb 11 07:27 db
drwxrwxrwx 2 hadoop hadoop 41 Feb 11 07:27 rules
[hadoop@ip-172-31-14-192 src]$
```

dao.py and geo_map.py – /home/hadoop/python/src/db

```
[hadoop@ip-172-31-14-192 db]$ ls -lrt
total 8
-rwxrwxrwx 1 hadoop hadoop 0 Feb 11 05:27 __init__.py
-rwxrwxrwx 1 hadoop hadoop 1205 Feb 11 05:27 geo_map.py
-rwxrwxrwx 1 hadoop hadoop 1168 Feb 11 05:27 dao.py
drwxrwxrwx 2 hadoop hadoop 63 Feb 11 07:27 __pycache__
```

rules.py - /home/Hadoop/python/src/rules

```
[hadoop@ip-172-31-14-192 db]$ cd ../rules/
[hadoop@ip-172-31-14-192 rules]$ ls -lrt
total 4
-rwxrwxrwx 1 hadoop hadoop 2715 Feb 11 05:27 rules.py
-rwxrwxrwx 1 hadoop hadoop 0 Feb 11 05:27 __init__.py
[hadoop@ip-172-31-14-192 rules]$
```

- Zip all the python dependencies into one file with name src.zip (containing dao.py, geo_map.py and rules.py in it).

```
zip src.zip __init__.py rules/* db/*
```

```
[hadoop@ip-172-31-11-213 src]$ zip src.zip __init__.py rules/* db/*
adding: __init__.py (stored 0%)
adding: rules/__init__.py (stored 0%)
adding: rules/__pycache__/ (stored 0%)
adding: rules/rules.py (deflated 68%)
adding: db/dao.py (deflated 61%)
adding: db/geo_map.py (deflated 56%)
adding: db/__init__.py (stored 0%)
adding: db/__pycache__/ (stored 0%)
[hadoop@ip-172-31-11-213 src]$
```

- Run the following command to enable Kafka Integration with Apache Spark.

```
export SPARK_KAFKA_VERSION=0.10
```

```
[hadoop@ip-172-31-11-213 src]$ export SPARK_KAFKA_VERSION=0.10
[hadoop@ip-172-31-11-213 src]$
```

- Run driver.py script to start spark streaming job which consumes real time transactions data and detects the status (FRAUD or GENUINE) and display the final output of each micro batches in the console.

```
spark-submit --py-files src.zip --packages
org.apache.spark:spark-sql-kafka-0-10_2.11:2.4.5 --files
uszipsv.csv driver.py
```

```

[logstash-172-31-11-213 root] spark-submit --packages org.apache.spark:spark-sql-kafka-0-10-2.11.2.4.5 --files usaspw.csw.driver.py /vy Default Cache set to : /home/hadoop/.ivy2/cache
The jars for the packages stored in : /home/hadoop/.ivy2/jars
Loading settings :: url = jar:file:/usr/lib/spark/jar/ivy-2.4.0.jar/org/apache/ivy/core/settings/ivysettings.xml
org.apache.spark:spark-sql-kafka-0-10-2.11 added as a dependency
:: resolving dependencies :: org.apache.spark:spark-submit-parent-a324ea9b-c6fa-4b34-b034-93164d56288:1.0
confs: [default]
found org.apache.spark:spark-sql-kafka-0-10-2.11.2.4.5 in central
found org.apache.kafka:kafka-clients-2.0.0 in central
found org.scala-lang:scala-2.10.4 in central
found org.merial.snappy#snappy-javal:1.7.3 in central
found org.aifc#aifc-april.7.16 in central
found org.spark-problem:statsmodels:1.4.0 in central
:: resolution report :: resolve 317ms :: artifacts 41 items
:: modules in use ::
org.apache.kafka:kafka-clients-2.0.0 from central in [default]
org.apache.spark:spark-sql-kafka-0-10-2.11.2.4.5 from central in [default]
org.scala-lang:scala-2.10.4 from central in [default]
org.aifc#aifc-april.7.16 from central in [default]
org.spark-problem:statsmodels:1.4.0 from central in [default]
org.merial.snappy#snappy-javal:1.7.3 from central in [default]
| | | | | | | | | | |
| conf | number | search(denied|evicted) | number(denied) |
| default | 6 | 0 | 0 | 0 | 0 | 1 | 4 | 0 | 1 |
:: retrieving :: org.apache.spark:spark-submit-parent-a324ea9b-c6fa-4b34-b034-93164d56288
confs: [default]
0 artifacts copied, 6 already retrieved (0KB/5ms)
[2022/11/09:24:41 INFO SparkContext: Running Spark version 2.4.5-amzn-0
[2022/11/09:24:41 INFO SparkContext: Submitted application: Fraud Detection - Card Transactions
[2022/11/09:24:41 INFO SecurityManager: Changing view acls to: hadoop
[2022/11/09:24:41 INFO SecurityManager: Changing modify acls to: hadoop
[2022/11/09:24:41 INFO SecurityManager: Changing view acls groups to:
[2022/11/09:24:41 INFO SecurityManager: Changing modify acls groups to:
[2022/11/09:24:41 INFO SecurityManager: SecurityManager: authentication disabled; ui acls disabled; users with view permissions: Set(hadoop); groups with view permissions: Set(); users with modify permissions: Set(hadoop)
modify permission Set()
[2022/11/09:24:42 INFO Utils: Successfully started service "SparkDriver" on port 43633.
[2022/11/09:24:42 INFO SparkEnv: Registering MapOutputTracker
[2022/11/09:24:42 INFO SparkEnv: Registering BlockManagerMaster
[2022/11/09:24:42 INFO BlockManagerMasterEndpoint: Using org.apache.spark.dynamicAllocation.DefaultTopologyMapper for getting topology information
[2022/11/09:24:42 INFO BlockManagerMasterEndpoint: BlockManagerMasterEndpoint up
[2022/11/09:24:42 INFO DiskBlockManager: Created local directory at /mnt/tmp/blockmgr-8baac193-1764-4485-8412-e1aff7f59195
[2022/11/09:24:42 INFO MemoryStore: MemoryStore started with capacity 1024 MB
[2022/11/09:24:42 INFO SparkEnv: Registering OutputCommitCoordinator
[2022/11/09:24:42 INFO Utils: Successfully started service "SparkUI" on port 4040.
[2022/11/09:24:42 INFO SparkUI: Bound SparkUI to 0.0.0.0 and started at http://172-31-11-213.ec2.internal:4040
[2022/11/09:24:43 INFO Client: Using initial executors = 100, max of spark.dynamicAllocation.initialExecutors, spark.dynamicAllocation.minExecutors and spark.executor.instances
[2022/11/09:24:43 INFO RMProxy: Connecting to ResourceManager at ip-172-31-11-213.ec2.internal/172.31.11.213:8032
[2022/11/09:24:43 INFO Client: Requesting a new application from cluster with 2 NodeManagers
[2022/11/09:24:43 INFO Client: Verifying our application has not requested more than the maximum memory capability of the cluster (6144 MB per container)
[2022/11/09:24:43 INFO Client: Will allocate AM container, with 596 MB memory including 384 MB overhead
[2022/11/09:24:43 INFO Client: Setting up container launch context for our AM
[2022/11/09:24:43 INFO Client: Setting up the launch environment for our AM container
[2022/11/09:24:43 INFO Client: Preparing resources for our AM container
[2022/11/09:24:43 INFO Client: Writing spark.warehouse file to spark.warehouse archive is set, falling back to uploading libraries under SPARK_HOME.
[2022/11/09:24:43 INFO Client: Uploading resource file:/mnt/tmp/spark-da334cd0-fdd6-4bcb-aaf6-9ecfd2a9631f/ spark libs_6854940a66670373832.zip -- hdfs://ip-172-31-11-213.ec2.internal:8020/users/hadoop/.sparkteagingspark-w

```

```

24/02/11 09:26:12 INFO Utils: Successfully started service 'org.apache.spark.network.netty.NettyBlockTransferService' on port 36763.
24/02/11 09:26:12 INFO NettyBlockTransferService: Service created on ip-172-31-11-213.ec2.internal:36763
24/02/11 09:26:12 INFO BlockManager: User-defined BlockManagerLocationInitialAllocationPolicy for Block replication policy
24/02/11 09:26:12 INFO BlockManagerMaster: Registering BlockManager BlockManagerId(driver, ip-172-31-11-213.ec2.internal, 36763, None)
24/02/11 09:26:12 INFO BlockManagerMaster: Registering block manager ip-172-31-11-213.ec2.internal:36763 with 1028.8 MB RAM, BlockManagerId(driver, ip-172-31-11-213.ec2.internal, 36763, None)
24/02/11 09:26:12 INFO BlockManagerMaster: Registered BlockManager BlockManagerId(driver, ip-172-31-11-213.ec2.internal, 36763, None)
24/02/11 09:26:12 INFO BlockManager: external shuffle service port = 7937
24/02/11 09:26:12 INFO BlockManager: Initialized BlockManager: BlockManagerId(driver, ip-172-31-11-213.ec2.internal, 36763, None)
24/02/11 09:26:12 INFO JettyUtils: Adding filter org.apache.hadoop.yarn.server.webproxy.amfilter.AmpFilter to /metrics/json.
24/02/11 09:26:12 INFO EventLoggingListener: Logging events to hdf://var/log/spark/apps/application_1707633848249_0021
24/02/11 09:26:12 INFO EventLoggingListener: Initial executors = 100, max of spark.dynamicAllocation.initialExecutors, spark.dynamicAllocation.minExecutors and spark.executor.instances
24/02/11 09:26:12 INFO YarnSchedulerBackend: SchedulerBackend is ready for scheduling beginning after reached minRegisteredResourcesRatio: 0.0
24/02/11 09:26:12 INFO ShareState: loading hive config file: file:/etc/spark/conf.d/hive-site.xml
24/02/11 09:26:12 INFO ShareState: Setting hive.metastore.warehouse.dir ('null') to the value of spark.sql.warehouse.dir ('hdfs:///user/spark/warehouse').
24/02/11 09:26:12 INFO ShareState: Warehouse path is 'hdfs:///user/spark/warehouse'.
24/02/11 09:26:12 INFO JettyUtils: Adding filter org.apache.hadoop.yarn.server.webproxy.amfilter.AmpFilter to /SQL.
24/02/11 09:26:12 INFO JettyUtils: Adding filter org.apache.hadoop.yarn.server.webproxy.amfilter.AmpFilter to /SQL/json.
24/02/11 09:26:12 INFO JettyUtils: Adding filter org.apache.hadoop.yarn.server.webproxy.amfilter.AmpFilter to /SQL/execution.
24/02/11 09:26:12 INFO JettyUtils: Adding filter org.apache.hadoop.yarn.server.webproxy.amfilter.AmpFilter to /SQL/execution/json.
24/02/11 09:26:12 INFO JettyUtils: Adding filter org.apache.hadoop.yarn.server.webproxy.amfilter.AmpFilter to /static/sql.
24/02/11 09:26:13 INFO StateStoreCoordinatorRef: Registered StateStoreCoordinator endpoint

```

```

Batch: 0

```

card_id	member_id	amount	pos_id	postcode	transaction_dt	status
14870230256514	137495066290	1438091212480340680722156774			2018-03-01 08:24:25	GENUINE
14870230256514	137495066290	167033851786562777140812184758			2018-06-02 04:15:03	GENUINE
14870230256514	137495066290	174543281466952571393508193645			2018-02-12 09:56:42	GENUINE
14870230256514	137495066290	40193428145845320308319118686			2018-06-13 05:38:54	GENUINE
14870230256514	137495066290	154583351544596211846391750033			2018-06-16 21:51:54	GENUINE
14870230256514	137495066290	13966214136526634272501122832			2018-10-21 03:52:51	GENUINE
14870230256514	137495066290	1753644194570529252671117623			2018-08-23 00:11:30	GENUINE
14870230256514	137495066290	165211276475551558660155708			2018-11-23 17:02:39	GENUINE
181896316803974	117826301530	19222134152570137355194164002			2018-03-01 20:22:10	GENUINE
181896316803974	117826301530	14133948118203138343115126346			2018-09-09 01:52:32	GENUINE
181896316803974	117826301530	189389217997426464101709334			2018-12-09 05:20:53	FRAUD
181896316803974	117826301530	1378636612176815071261643431			2018-08-12 14:29:30	GENUINE
181896316803974	117826301530	1914223715642059678903150635			2018-06-16 19:37:19	GENUINE
1540707334486644	1147822084344	6885448188791390671111159031			2018-05-05 07:53:53	GENUINE
1540707334486644	1147822084344	4028209116266605118182180118			2018-08-11 01:06:50	GENUINE
1540707334486644	1147822084344	3383648186410551703328153920			2018-07-12 17:37:26	GENUINE
1540707334486644	1147822084344	930773317293741601647814989			2018-07-13 04:50:16	GENUINE
1540707334486644	1147822084344	4011296154337336731964744028			2018-10-17 13:09:34	GENUINE
1540707334486644	1147822084344	3453321150565659311145453			2018-04-21 14:12:26	FRAUD
1540707334486644	1147822084344	75500741345533088112095115030			2018-09-29 02:34:52	FRAUD

HBase:
count 'card transactions'

Before Streaming – 53292 records

```
=> ["card_lookup", "card_transactions"]
hbase(main):002:0> count 'card_transactions'
Current count: 1000, row: 341719092861087~348479157173653~09-12-2017 01:50:04~5777101.0
Current count: 2000, row: 343252340741735~878481175923264~17-03-2016 01:27:55~499306.0
Current count: 3000, row: 344453305077666~477889498690453~03-05-2017 13:17:03~4112459.0
Current count: 4000, row: 345864096406128~263597298380498~10-01-2018 13:19:13~5620846.0
Current count: 5000, row: 347177017703006~607111050994623~24-11-2017 19:29:35~3881266.0
Current count: 6000, row: 348390084708787~252122735905449~31-08-2017 15:58:36~4365570.0
Current count: 7000, row: 349261557797449~220369360742912~10-11-2017 13:09:04~4130551.0
Current count: 8000, row: 371085417506954~790236765996611~22-08-2016 15:36:09~3667024.0
Current count: 9000, row: 372838963275954~191667371054185~06-04-2017 08:12:05~5693002.0
Current count: 10000, row: 374437449333250~104804337714838~12-11-2016 17:23:01~9084333.0
Current count: 11000, row: 375956299108731~733657582169915~10-08-2017 23:08:32~9766446.0
Current count: 12000, row: 377037030364630~605988874514427~06-10-2017 12:13:10~2332892.0
Current count: 13000, row: 378118026791597~586336168666572~25-08-2017 19:31:43~1280928.0
Current count: 14000, row: 4009218272111551~298935629685938~01-08-2016 22:46:05~8656880.0
Current count: 15000, row: 4067184730430984~275178801561140~21-02-2017 14:21:14~6830136.0
Current count: 16000, row: 4132381122041426~598314604248219~10-12-2016 21:17:21~9303987.0
Current count: 17000, row: 4264553579186587~858567976578612~18-04-2017 23:55:12~6058344.0
Current count: 18000, row: 4373464339970856~657196939516344~11-01-2018 00:00:00~6559716.0
Current count: 19000, row: 4439501833420177~634293040387939~17-02-2017 05:01:05~5371581.0
Current count: 20000, row: 4500499651579063~277121920702488~08-12-2017 13:32:44~3513696.0
Current count: 21000, row: 4566853351752365~335349979924073~03-04-2017 20:30:40~3669382.0
Current count: 22000, row: 4600996294769125~259895316688347~25-12-2017 00:33:18~7070746.0
Current count: 23000, row: 4689314809377828~989600253556389~24-10-2016 02:52:59~185332.0
Current count: 24000, row: 4766789897935106~072938202866006~09-10-2017 20:58:38~4206082.0
Current count: 25000, row: 4863127030291206~483187165654021~12-05-2016 23:19:14~3878839.0
Current count: 26000, row: 4907253800863053~776565941881281~14-12-2017 09:59:35~7434309.0
Current count: 27000, row: 4995478705000641~155223326542744~20-05-2017 02:02:58~1780486.0
Current count: 28000, row: 5134479292018417~128069614660231~08-11-2017 08:29:00~4083279.0
Current count: 29000, row: 5162808285745682~924897469747000~02-07-2016 15:57:01~1888498.0
Current count: 30000, row: 5186615811954262~026744451387425~23-01-2018 06:47:40~2972350.0
Current count: 31000, row: 5221229593682054~127498352656607~16-08-2017 16:04:44~8870294.0
Current count: 32000, row: 5258095619226135~524892536444302~11-11-2017 00:00:00~5577468.0
Current count: 33000, row: 5294592751808411~698043686524445~28-12-2017 19:50:46~792320.0
Current count: 34000, row: 5342400571435088~569800919443173~26-08-2016 07:22:56~2049055.0
Current count: 35000, row: 5380978184175608~667233655872426~01-09-2016 10:51:09~9057468.0
Current count: 36000, row: 5414439899219272~345244546468970~07-06-2017 00:10:15~1413268.0
Current count: 37000, row: 5481808794715436~234167732114687~20-10-2017 16:48:20~3012181.0
Current count: 38000, row: 5534323829711423~637125777249291~30-06-2017 11:43:05~535263.0
Current count: 39000, row: 5584977018799504~770952296643375~03-05-2017 09:46:23~9635501.0
Current count: 40000, row: 6011139413319542~936287997923127~30-03-2016 07:13:43~1908818.0
Current count: 41000, row: 6011525010455848~959531034098755~02-08-2017 23:46:55~2762867.0
Current count: 42000, row: 6011782857327719~167569302823461~09-10-2016 02:59:08~8187310.0
Current count: 43000, row: 6221796595498984~174952353920145~18-02-2016 01:04:16~7689490.0
Current count: 44000, row: 6224271253849917~048019406374284~12-12-2017 19:22:13~4960131.0
Current count: 45000, row: 6225606551069826~648178895653883~13-07-2017 13:58:12~4112360.0
Current count: 46000, row: 6228733641419063~422091580968713~11-05-2016 10:27:11~2641410.0
Current count: 47000, row: 6447877814927926~992747968210744~11-01-2018 00:00:00~9774152.0
Current count: 48000, row: 6461356425954109~839304530643246~03-11-2016 20:57:23~1877568.0
Current count: 49000, row: 6480152634975473~963893207999520~17-01-2018 17:22:08~5863997.0
Current count: 50000, row: 6505080237250161~874636482279942~18-09-2017 17:36:33~2156682.0
Current count: 51000, row: 6544876671165176~424648847023528~04-08-2017 16:09:39~5155240.0
Current count: 52000, row: 6574255180086418~504711588075355~20-12-2017 08:11:48~5899422.0
Current count: 53000, row: 6595814135833988~311764663134170~18-11-2017 12:30:42~2717739.0
53292 row(s) in 2.9170 seconds

=> 53292
```

After Streaming – 59375 records

```

Current count: 3000, row: 345864096406128~263597298380498~10-01-2018 13:19:13~5620846.0
Current count: 4000, row: 347177017703006~607111050994623~24-11-2017 19:29:35~3881266.0
Current count: 5000, row: 348390084708787~252122735905449~31-08-2017 15:58:36~4365570.0
Current count: 6000, row: 349261557797449~220369360742912~10-11-2017 13:09:04~4130551.0
Current count: 7000, row: 371085417506954~790236765996611~22-08-2016 15:36:09~3667024.0
Current count: 8000, row: 372838963275954~191667371054185~06-04-2017 08:12:05~5693002.0
Current count: 9000, row: 374437449333250~104804337714838~12-11-2016 17:23:01~9084333.0
Current count: 10000, row: 375956299108731~733657582169915~10-08-2017 23:08:32~9766446.0
Current count: 11000, row: 377037030364630~605988874514427~06-10-2017 12:13:10~2332892.0
Current count: 12000, row: 378118026791597~586336168666572~25-08-2017 19:31:43~1280928.0
Current count: 13000, row: 4009218272111551~298935629685938~01-08-2016 22:46:05~8656880.0
Current count: 14000, row: 4067184730430984~275178801561140~21-02-2017 14:21:14~6830136.0
Current count: 15000, row: 4132381122041426~598314604248219~10-12-2016 21:17:21~9303987.0
Current count: 16000, row: 4264553579186587~858567976578612~18-04-2017 23:55:12~6058344.0
Current count: 17000, row: 4373464339970856~657196939516344~11-01-2018 00:00:00~6559716.0
Current count: 18000, row: 4439501833420177~634293040387939~17-02-2017 05:01:05~5371581.0
Current count: 19000, row: 4500499651579063~277121920702488~08-12-2017 13:32:44~3513696.0
Current count: 20000, row: 4566853351752365~335349979924073~03-04-2017 20:30:40~3669382.0
Current count: 21000, row: 4600996294769125~259895316688347~25-12-2017 00:33:18~7070746.0
Current count: 22000, row: 4689314809377828~989600253556389~24-10-2016 02:52:59~185332.0
Current count: 23000, row: 4766789897935106~072938202866006~09-10-2017 20:58:38~4206082.0
Current count: 24000, row: 4863127030291206~483187165654021~12-05-2016 23:19:14~3878839.0
Current count: 25000, row: 4907253800863053~776565941881281~14-12-2017 09:59:35~7434309.0
Current count: 26000, row: 4995478705000641~155223326542744~20-05-2017 02:02:58~1780486.0
Current count: 27000, row: 5134479292018417~128069614660231~08-11-2017 08:29:00~4083279.0
Current count: 28000, row: 5162808285745682~924897469747000~02-07-2016 15:57:01~1888498.0
Current count: 29000, row: 5186615811954262~026744451387425~23-01-2018 06:47:40~2972350.0
Current count: 30000, row: 5221229593682054~127498352656607~16-08-2017 16:04:44~8870294.0
Current count: 31000, row: 5258095619226135~524892536444302~11-11-2017 00:00:00~5577468.0
Current count: 32000, row: 5294592751808411~698043686524445~28-12-2017 19:50:46~792320.0
Current count: 33000, row: 5342400571435088~569800919443173~26-08-2016 07:22:56~2049055.0
Current count: 34000, row: 5380978184175608~667233655872426~01-09-2016 10:51:09~9057468.0
Current count: 35000, row: 5414439899219272~345244546468970~07-06-2017 00:10:15~1413268.0
Current count: 36000, row: 5481808794715436~234167732114687~20-10-2017 16:48:20~3012181.0
Current count: 37000, row: 5534323829711423~637125777249291~30-06-2017 11:43:05~535263.0
Current count: 38000, row: 5584977018799504~770952296643375~03-05-2017 09:46:23~9635501.0
Current count: 39000, row: 6011139413319542~936287997923127~30-03-2016 07:13:43~1908818.0
Current count: 40000, row: 6011525010455848~959531034098755~02-08-2017 23:46:55~2762867.0
Current count: 41000, row: 6011782857327719~167569302823461~09-10-2016 02:59:08~8187310.0
Current count: 42000, row: 6221796595498984~174952353920145~18-02-2016 01:04:16~7689490.0
Current count: 43000, row: 6224271253849917~048019406374284~12-12-2017 19:22:13~4960131.0
Current count: 44000, row: 6225606551069826~648178895653883~13-07-2017 13:58:12~4112360.0
Current count: 45000, row: 6228733641419063~422091580968713~11-05-2016 10:27:11~2641410.0
Current count: 46000, row: 6447877814927926~992747968210744~11-01-2018 00:00:00~9774152.0
Current count: 47000, row: 6461356425954109~839304530643246~03-11-2016 20:57:23~1877568.0
Current count: 48000, row: 6480152634975473~963893207999520~17-01-2018 17:22:08~5863997.0
Current count: 49000, row: 6505080237250161~874636482279942~18-09-2017 17:36:33~2156682.0
Current count: 50000, row: 6544876671165176~424648847023528~04-08-2017 16:09:39~5155240.0
Current count: 51000, row: 6574255180086418~504711588075355~20-12-2017 08:11:48~5899422.0
Current count: 52000, row: 6595814135833988~311764663134170~18-11-2017 11:13:42~2717739.0
Current count: 53000, row: 6565814135833988~311764663134170~18-11-2017 19:21:12~2717739.0
Current count: 54000, row: 66500814135833988~311764663134170~18-11-2017 06:42:22~43471739.0
Current count: 55000, row: 6623581435833988~311764663134170~18-11-2017 01:23:22~2227739.0
Current count: 56000, row: 670014135833988~311764663134170~18-11-2017 02:12:12~1137739.0
Current count: 57000, row: 68486814135833988~311764663134170~18-11-2017 22:11:23~2744439.0
Current count: 58000, row: 6895448135833988~311764663134170~18-11-2017 21:20:42~2717739.0
59375 row(s) in 3.12360 seconds

=> 59375
hbase(main):003:0>

[root@ip-10-0-7-7 ~]#

```

Data validation

Hive:

```
select status,count(*) from card_transactions group by status;
```

Before Streaming:

- FRAUD 82
- GENUINE 53210

```
[root@ip-172-31-14-192 ~]# hive

Logging initialized using configuration in file:/etc/hive/conf.dist/hive-log4j2.properties Async: true
hive> show tables;
OK
card_lookup
card_member
card_transactions
card_transactions_stg
last_ten_transactions
member_score
Time taken: 0.551 seconds, Fetched: 6 row(s)
hive> select status, count(*) from card_transactions group by status;
Query ID = root_20240211073550_05c811fa-3d40-4f68-962b-125bbcef4659
Total jobs = 1
Launching Job 1 out of 1
Status: Running (Executing on YARN cluster with App id application_1707635973321_0007)

-----
VERTICES      MODE           STATUS  TOTAL  COMPLETED  RUNNING  PENDING  FAILED  KILLED
-----
Map 1 ..... container    SUCCEEDED    1         1         0         0         0         0
Reducer 2 ..... container    SUCCEEDED    2         2         0         0         0         0
-----
VERTICES: 02/02  [=====>>>] 100%  ELAPSED TIME: 7.93 s
-----
OK
FRAUD      82
GENUINE 53210
Time taken: 17.193 seconds, Fetched: 2 row(s)
hive> █
```

After Streaming:

- FRAUD 211
- GENUINE 59145

```
hive> select status, count(*) from card_transactions group by status;
Query ID = root_20240211103520_a9173084-8446-4574-alad-cd4856f47338
Total jobs = 1
Launching Job 1 out of 1
Status: Running (Executing on YARN cluster with App id application_1707646699281_0006)
```

VERTICES	MODE	STATUS	TOTAL	COMPLETED	RUNNING	PENDING	FAILED	KILLED
Map 1	container	SUCCEEDED	1	1	0	0	0	0
Reducer 2	container	SUCCEEDED	2	2	0	0	0	0

```
VERTICES: 02/02 [=====>>] 100% ELAPSED TIME: 7.48 s
OK
FRAUD 211
GENUINE 59145
Time taken: 16.624 seconds, Fetched: 2 row(s)
hive>
```

- Check the latest transaction date available in card_transactions table.

Hive:

```
select max(transaction_dt) from card_transactions;
```

Before Streaming: 31-12-2017 23:20:13

```
hive> select max(transaction_dt) from card_transactions;
Query ID = root_20240211104235_d37cd312-af4a-4ae6-b66a-c8a9d5fbbbf3
Total jobs = 1
Launching Job 1 out of 1
Status: Running (Executing on YARN cluster with App id application_1707646699281_0007)
```

VERTICES	MODE	STATUS	TOTAL	COMPLETED	RUNNING	PENDING	FAILED	KILLED
Map 1	container	SUCCEEDED	1	1	0	0	0	0
Reducer 2	container	SUCCEEDED	1	1	0	0	0	0

```
VERTICES: 02/02 [=====>>] 100% ELAPSED TIME: 6.99 s
OK
31-12-2017 23:20:13
Time taken: 14.971 seconds, Fetched: 1 row(s)
```

After Streaming: 11-02-2018 12:10:33

```
hive> select max(transaction_dt) from card_transactions;
Query ID = root_20240211104235_d37cd312-af4a-4ae6-b66a-c8a9d5fbbbf3
Total jobs = 1
Launching Job 1 out of 1
Status: Running (Executing on YARN cluster with App id application_1707646699281_0007)
```

VERTICES	MODE	STATUS	TOTAL	COMPLETED	RUNNING	PENDING	FAILED	KILLED
Map 1	container	SUCCEEDED	1	1	0	0	0	0
Reducer 2	container	SUCCEEDED	1	1	0	0	0	0

```
VERTICES: 02/02 [=====>>] 100% ELAPSED TIME: 6.99 s
OK
11-02-2018 12:10:33
Time taken: 16.121 seconds, Fetched: 1 row(s)
```